

订单生命周期：从下单到结清的 7 个业务节点

C 端 / 商家 / 客服 / 运营 / SRE 各看到什么—gomall v2.2 (business-first)

gomall · 业务侧 deck

今日大纲

业务困局：一笔订单要回答 5 个角色的问题

7 态 × 3 视角：C 端 / 商家 / 客服各看到什么

关单博弈：30min 倒计时背后的业务取舍

客服话术帧：业务码 + SOP 一一对照

业务承诺总表与 SLO

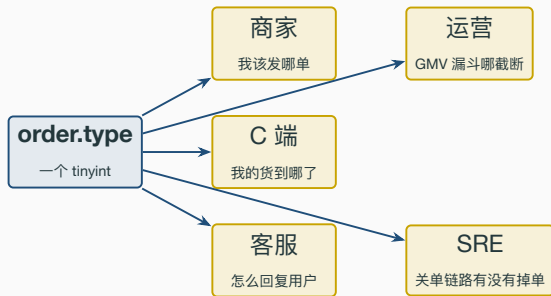
状态机实现：3 层兜底（业务规则 / SQL / 事件）

业务边界：deck 09 不做什么（履约 gap 路线图）

兼容性：3 → 7 零停机迁移

业务困局：一笔订单要回答 5 个角色的问题

订单状态不是 type 字段，是 5 个角色的协同语言



- 一个 type 字段同时是 **C 端跟踪页面文案**、**商家工作台筛选条件**、**客服 SOP 入口**、**运营漏斗节点**、**SRE 关单监控指标**。
- 状态切得不够细，5 个角色就要在工单 / IM / 群里口头补一这是真正的业务债。
- gomall 把 type 拆到 7 态，是给这 5 个角色一个统一的事实表 (single source of truth)。

3 态够吗：旧版本卡住的真实业务症状

- 旧版本只有 UnPaid / Paid / Cancelled 三态，C 端付了款只能看到“已支付”，但货还没发。
- 客服日均高频客诉 #1：我付了 3 天还没发货，你们是不是跑路了。客服只能 IM 问商家，回话延迟在小时级。
- 商家工作台无筛选条件：商家不知道哪些是待发货、哪些是用户已收货，每天靠人肉 sheet 维护。
- 运营漏斗失真：付款 → 发货 → 收货三个节点合并成“Paid”，无法算 发货时效 / 签收时效 / 自动确认率。
- 售后 / 退款无落点：用户申请退款无独立态，业务方只能在 Paid 上加 flag，越加越脏。

业务侧的状态划分粒度 = 角色之间需要协同的最小事件。3 态合 5 件事，必爆。

gomall v2.2 的 7 态决策：业界对齐 + 业务诉求

- 7 态 = 业界（淘宝 / 京东 / 拼多多 / Amazon）共有的最低公约数，做小要回头补、做大需要部分发货 / 多 SKU 子单。
- 数值兼容：1/2/3 沿用旧值（存量数据零迁移），4-7 是新增；旧名 UnPaid / Paid / Cancelled 曾以 Deprecated 别名过渡，调用方迁移完成后已删除，现只剩 OrderWaitPay 等现行命名。

7 态 × 3 视角：C 端 / 商家 / 客服各看到什么

帧 1 · WaitPay: C 端 / 商家 / 客服各看到什么

- 业务关键: **30min 倒计时**是 C 端最敏感的体验点—太短伤”研究着买”用户, 太长锁库存伤商家。
- 商家故意看不到 WaitPay 订单—没付钱的不算”待发货工作量”, 否则商家工作台数据失真。
- 客服 SOP: 用户问”为什么 30min 关单”→ 引用平台规则 + 引导重下; 用户付款失败 → 排查支付通道。

帧 2 · WaitShip: 商家工作流的真正起点

- 客服客诉 #2: 我付款 3 天显示待发货。客服界面能看到付款时间戳 + 商家最近登录, 秒回 SOP。
- 业务承诺: 商家 **48h 未发货** → 用户可一键申请退款 (路线图: 超时自动拉到 Refunding)。
- 平台 **P0 指标**: 付款 → 发货的 p95 时延 = 平台对商家的硬约束。

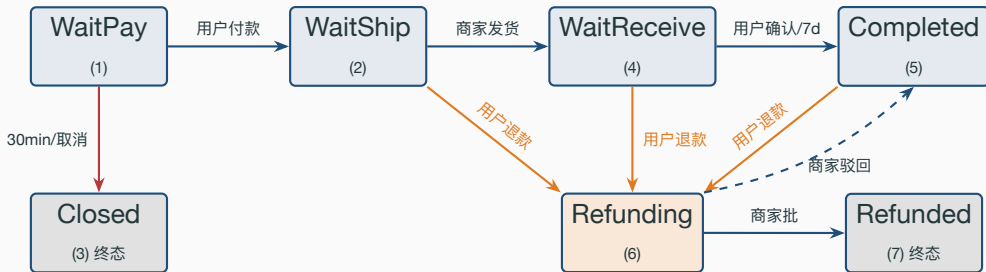
帧 3 · WaitReceive: 平台 7d 自动确认的资金钟

- 业务事实: 绝大多数用户不主动点确认收货—收到货之后没人会专门回 App 点一下。
- 平台 cron 每 6 小时扫一次 `updated_at > 7d` 的 WaitReceive 订单 → 自动推进到 Completed。
- 这个动作是资金结算给商家的触发器: 商家货款 = Completed 后 T+1 入账 (路线图)。

帧 4 · Completed / Closed / Refunding / Refunded: 三终态 + 一中间

- **Refunding** 是唯一可双向出边的中间态：批准 → Refunded, 驳回 → Completed (评价重开 / 售后工单关)。
- **Closed** 不进商家工作台：未付款的不算业务量；C 端可看到，方便引导”重新下单”挽回 GMV。
- **Completed** 不是死刑：售后窗口期内仍可发起退款 (Completed → Refunding 这条边存在)。

完整状态机：用户 / 商家 / 平台各自的边



- **蓝边** = 正向流转（业务期望）；**橙边** = 用户触发退款；**灰边** = 关单。
- 每条边都有**触发主语**（用户 / 商家 / cron）：业务上不允许商家替用户确认收货（防薅平台羊毛）。

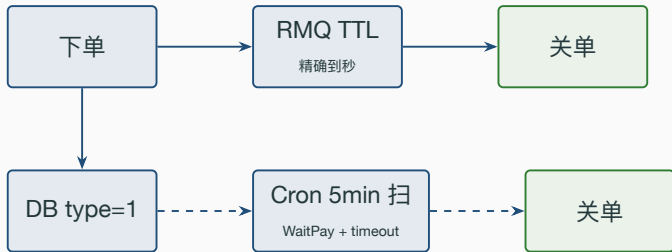
关单博弈：30min 倒计时背后的业务取舍

30min 倒计时：体验 vs 锁库存的钢丝绳



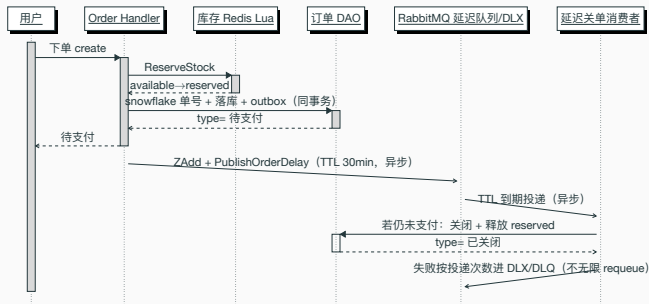
- **业务事实：** 用户加购到付款的中位时长 5-8min（业界公开数据），**30min** = 给 95%+ 用户足够时间 + 给商家可控的库存锁定窗口。
- **平台四方对齐：** 淘宝 30min / 京东 30min / 拼多多 30min / Amazon 30min — gomall 对齐业界。
- **业务规则：** 超时关单同时释放 **Redis reserved** 预占库存，让下一批用户能正常下单。

为什么单一保险不够：RMQ 漏 + Cron 慢



- **单 RMQ 不够**：RMQ 重启 / 消息丢失 / 消费者挂掉 = 漏关一单 = **库存永久锁死**。
- **单 Cron 不够**：5min 跑一次 → 实时性差，用户付款失败后看着“待付款”挂十几分钟。
- **双保险叙事**：RMQ 走**快线**（精确到秒），Cron 走**慢线**（兜底）一漏一个不漏单。
- **业务承诺**：**不允许漏关一单**。两条路共享同一个 CancelUnpaidOrder 入口。

时序图：下单落库 + 超时延迟关单的完整交互



- 下单同步段：预扣库存 + snowflake 单号 + 落库 + 写 outbox 一个事务收口，对外只产出”待支付”。
- 延迟关单段：RabbitMQ 走 **TTL 延迟快线**，到期投关单消费者；仍未支付则置”已关闭”并释放 reserved 预占。
- 双保险 + 毒丸隔离：**RMQ TTL 延迟 + Cron 兜底**共享同一关单入口；坏消息按投递次数进 **DLQ**，不无限 requeue。

毒丸消息：失败关单无限 requeue 把延迟队列打爆

```
87 if err := handler(orderNum); err != nil {
88     util.LogrusObj.Errorf("处理延迟关单失败 orderNum=%d err=%v\n", orderNum, err)
89     // order.dead.queue 未配 DLX, requeue 会立即回到队首重投。
90     // 仅允许首次失败重投一次；再次失败 (Redelivered) 转独立死信队列，
91     // 避免持续失败的毒丸消息无界回灌、占满消费者。
92     if d.Redelivered {
93         RouteToDLQ(d, orderDeadQueue, orderDeadRouting, false)
94         return
95     }
96     _ = d.Nack(false, true)
97     return
98 }
```

Listing 1: repository/rabbitmq/order_delay.go:87-98 失败消息分流

- 修前：handler 失败一律 Nack(requeue=true) ——一条 CloseOrderWithCheck 反复失败的订单（如关联 outbox 写库持续报错）回到队首立刻重投，形成热循环。
- 业务风险：单条坏消息 100% CPU 空转重投，把消费者吃满；后面排队的正常超时订单被饿死，30min 关单 SLA 失守、库存预占释放被卡住。
- 修后：首投失败仍重试一次（吸收瞬时抖动）；判定 Redelivered 即超阈值，RouteToDLQ 显式投独立死信队列并 Ack，原队列不再回灌。
- 死信里的坏单留给运维人工补偿，正常关单流量不被一条毒丸拖垮。

已修复复盘：cron 表达式陷阱与现行写法

```
19 c := cron.New(cron.WithSeconds())
20 orderService := new(order.OrderTaskService)
21 // 现行: @every 5m, 每 5min 整体触发一次 (语义明确、不踩秒位陷阱)
22 // 对照陷阱: "*/5 * * * *" 在 WithSeconds 下 = 秒位每秒 + 分位每 5min
23 // = 实际 60x/5min, 是这类六段表达式最常见的误写
24 _, err := c.AddFunc("@every 5m", func() {
25     defer func() {
26         if r := recover(); r != nil {
27             util.LogrusObj.Errorf("Cron 任务发生 Panic: %v", r)
28         }
29     }()
30     orderService.RunOrderTimeoutCheck()
31 })
```

Listing 2: initialize/cron.go:19-28 现行写法 @every 5m (左侧为陷阱对照)

- 陷阱本身: `*/5 * * * *` 在 `WithSeconds` 下读作秒位 = 任意 (每秒) + 分位每 5 min 一行业里最常踩的 **60x/5min** 误写。
- 现行做法: 用 `@every 5m` 取代六段表达式, 语义一眼可读, 从源头避开秒位歧义。
- 若误写会有代价: DB 每分钟扫一次 `WHERE type=1`, 6M 行 `order` 表单峰压力 60x; `outbox order.cancelled` 写入量虚高让 SRE 误判关单暴涨。
- 为什么即便误写也不会超卖: `CloseOrderWithCheck` 的 `WHERE type=1` 条件 `UPDATE` 天然幂等, 重复扫只是 `RowsAffected=0` 空跑—正确性靠幂等兜兜住, 频率正确性靠表达式选型。

cron 陷阱复盘：误写代价与现行加固 i

误写代价：DB 压力 60×

SELECT 全表扫

误写代价：日志洪水

LogrusObj 60×

现行：

"@every 5m"

误写代价：监控失真

outbox 写入计数

误写代价：潜在误关

边界并发

- 当初的可见症状：dev 环境 DB CPU 周期性飙红 + logrus 日志量异常 → 引出排查并修复。
- 为什么始终没造成事故：幂等兜底 + 关单不可逆 + 释放 reserved 也是 IF reserved ≥ N THEN ... 的 Lua 幂等。
- 现行加固：表达式改用 @every 5m（每 5min 整体触发 1 次），从写法上消除秒位歧义。
- 为什么 deck 要保留这段复盘：把“踩过的坑 + 怎么收口”讲清楚——在面试 / CR 中比“假装全部一帆风顺”更值钱。

幂等设计的隐藏 ROI：它不仅防止业务超卖，也吸收了重启重跑、网络重试乃至当初的表达式误写——但频率正确性仍须靠表达式选型本身保证。

CancelUnpaidOrder: 4 个调用方共享一个幂等入口

```
19 func CancelUnpaidOrder(orderNum uint64) error {
20     ctx := context.Background()
21     baseDao := NewOrderDao(ctx)
22     order, err := baseDao.GetOrderByOrderNum(orderNum)
23     if err != nil { return err }
24     if order == nil || order.ID == 0 { return nil }
25
26     var closed bool
27     err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
28         ok, err := NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
29         if err != nil { return err }
30         if !ok { return nil } // 已经被别的路径关掉了, no-op
31         closed = true
32         return outbox.NewOutboxDaoByDB(tx).Insert(
33             "order", "OrderCancelled", "order.cancelled", order.ID,
34             events.OrderCancelled{ /* ... */ Reason: "timeout",
35                 PromoRuleID: order.PromoRuleID,
36                 PromoDiscountCents: order.PromoDiscountCents})
37     })
38     if err != nil { return err }
39     if !closed { return nil }
40     if relErr := cache.ReleaseReservation(ctx,
41         order.ProductID, int64(order.Num)); relErr != nil {
42         util.LogrusObj.Errorf("release on cancel failed: %v", relErr)
43     }
44     return nil // 满减预算退还由 promo 侧消费事件异步完成
45 }
```

Listing 3: internal/order/cancel.go:19-67 共享入口 (节选)

- 4 个调用方: RMQ 消费者 / Cron 兜底 / 客服后台 / 用户主动取消—共享同一个入口、同一套兜底。
- closed=false 直接 no-op, 避免重复释放 reserved、避免重复写 outbox。
- 满减预算不在这里同步退: order.cancelled 事件载荷自带 promo_rule_id / promo_discount_cents, promo 侧 promo.release 队列消费后退预算, promo_release 台账 (OrderID 唯一索引) 吸收重复投递。

客服话术帧：业务码 + SOP 一一对照

客诉 #1: 30min 自动关单一客服怎么回

- **用户原话:** 我下单 30 分钟没付款, 订单怎么自动取消了? 我不知道还有时间限制。
- **客服 SOP:**
 - 1. 道歉 + 解释平台规则 (统一文案库): 为了保证库存周转, 所有订单付款窗口为 30 分钟。
 - 2. 引导重新下单 + 给 **5 元代金券** (路线图: 客服权限发券) 补偿。
 - 3. 后台核对: `order.type=3` + `reason=timeout` + `cancel_time` 在 `outbox` `order.cancelled` 事件里能查到。
- **业务码不暴露给客服:** 客服界面显示中文“已超时关闭”, 不显示 `type=3`。
- **留痕:** 客诉对话 + 补偿券发放都写工单, 运营复盘超时关单率。

客诉 #2: 付了 3 天没发货—客服怎么回

- 用户原话: 我付了 3 天了订单还在”待发货”, 是不是你们跑路了? 我要退款!
- 客服 SOP:
 - 1. 查商家工作台【待发货】tab 待处理数 + 商家最近登录时间。
 - 2. 商家 24h 内有活动 → IM 催商家, 挂**客服跟进工单** (4h 复查)。
 - 3. 商家 48h 无响应 → **客服代发起退款** (走 RequestRefund 调用, from=WaitShip 合法)。
 - 4. 客服 6h 内**先于商家批准退款** (避免用户二次客诉)。
- 平台承诺: 商家 48h 未发货, 用户一键退款 (路线图); 现版本靠客服主动 trigger。

客诉 #3: 点了确认收货能撤回吗—客服怎么回

- 用户原话：我刚点确认收货，发现东西有问题，能撤回吗？
- 客服 **SOP**:
 - 1. 不能撤回—状态机不允许 Completed → WaitReceive 回退（业务规则：避免薅羊毛）。
 - 2. 但是 Completed 不是死刑—售后期内仍可发起退款（Completed → Refunding 这条边存在）。
 - 3. 引导用户走售后申请：POST /orders/refund/request，理由选”商品质量问题”。
 - 4. 客服后台一键加急工单（路线图：售后工单服务落地后挂”加急” flag）。
- 为什么不允许撤回：用户撤回 = 资金回退商家结算 = T+1 入账的钱已经在路上 → 资金链反推不可行。

客诉 #4: 用户说没收到货—客服怎么回

- 用户原话: 我看物流显示签收了, 但我没收到货!
- 客服 **SOP**:
 - 1. 查物流单详情 (路线图: logistics_order 子表) + 签收人 / 签收时间 / 签收地址。
 - 2. 引导用户联系小区 / 代收点 / 家人 24h 内回查。
 - 3. 仍找不到货 → 客服代发起**全额退款 + 物流商索赔**流程 (路线图: 物流 webhook 联动)。
 - 4. 订单仍在 WaitReceive (用户没点确认) → 直接 RequestRefund。
 - 5. 订单已到 Completed (7d 自动) → 走售后期申诉, 靠 Completed → Refunding 边推进。

客诉 #5: 抢购 / 支付出现 70001 / 60002 — 客服怎么回

- **70001 限流** (ErrRateLimitExceeded): 您操作过于频繁, 请稍后再试。
 - 客服话术: 秒杀场景 1s 内 3 次以上点击会触发; 引导用户等 **1 秒**再点。
 - 不要解释”滑动窗口算法”, 用户只关心”我是不是被封号了” — 答案是没有, 仅限频。
- **60002 幂等处理中** (ErrIdempotencyInProgress): 订单正在处理, 请勿重复提交。
 - 用户在网络抖动时连点了 N 次 /orders/create, 第二次起返回 60002。
 - 客服话术: 您的订单已提交, 请回首页”我的订单”刷新查看。
- **70002 熔断打开** (ErrCircuitOpen): 当前业务繁忙, 请稍后再试。
 - 下游服务大面积失败被熔断, 客服关注**波及范围** → 转 SRE。

业务承诺总表与 SLO

业务承诺总表：7 条边 7 个业务事件

- 7 条承诺 = 7 态状态机的核心；每条对应一条 SQL（条件 UPDATE）+ 一个 outbox 事件 + 一份单测。
- **业务确定性的来源：**每条边的”主语”是固定的一用户不能替商家发货，商家不能替用户确认收货。

订单链路 SLO：从 README 业务承诺到压测对照

- 宁可下游慢、不能上游 429：下单 / 列表都不挂限流，靠缓存 + 异步削峰扛峰值。
- 留 5-10× 余量给 GC / 网络抖动：实测 $RPS \times 10 =$ 业务平时峰值；压测验证容量 ceiling。

业务可观测指标：5 个状态切换的 metrics

- 每条边都打 **metric**: `order_state_transition_total{from,to,trigger}`
——由 outbox publisher 消费事件落 Prometheus。
- 运营看板核心面板：
 - GMV 漏斗: WaitPay 创建 → WaitShip 转化率 → WaitReceive 转化率 → Completed 转化率
 - 关单率: $\text{Closed} / (\text{Closed} + \text{WaitShip})$ 按小时切分 (异常 → 支付通道 / 反欺诈警报)
 - 发货 p95 时延: $\text{WaitShip.updated_at} - \text{WaitShip.created_at}$
 - 自动确认占比: $\text{Completed.Auto=true} / \text{Completed.total}$ (应 >80% 业内基线)
 - 退款率: $\text{Refunded} / \text{Completed}$ (>3% 触发告警, 路线图)
- **SRE 告警**: 关单 RPS 突增 (误关风险) / 自动确认率突降 (cron 失活) / 退款率突增 (商品问题 / 黑产)。

**状态机实现：3 层兜底（业务规则 /
SQL / 事件）**

第 1 层：状态机表是业务规则，不是数据结构

```
21 var orderStateTransitions = map[uint][]uint{
22     consts.OrderWaitPay: {consts.OrderWaitShip, consts.OrderClosed},
23     consts.OrderWaitShip: {consts.OrderWaitReceive, consts.OrderRefunding},
24     consts.OrderWaitReceive: {consts.OrderCompleted, consts.OrderRefunding},
25     consts.OrderCompleted: {consts.OrderRefunding},
26     consts.OrderRefunding: {consts.OrderRefunded, consts.OrderCompleted},
27     consts.OrderWaitGroup: {consts.OrderWaitShip, consts.OrderClosed}, // 拼团过渡态 (deck 14)
28 }
```

Listing 4: internal/order/state.go:21-28 转换表

- **key** = 当前状态，**value** = 允许去的下一态集合——不在表里就是业务上非法。
- **Closed** 和 **Refunded** 不在 key 集合 → `IsTerminalOrderState` 直接 `true`。
- 为什么放 **service** 不放 **model**：状态机是业务规则，DB 只兜底“原子推进”，业务正确性靠这张表 + 单测。
- 17 例单测覆盖：合法 / 非法 / 终态 / 未知输入。

第 2 层：DAO 条件 UPDATE 是并发的最后一道闸

```
215 func (d *OrderDao) ShipOrder(orderNum uint64) (bool, error) {
216     res := d.DB.Model(&Order{}).
217         Where("order_num=? AND type=?", orderNum, consts.OrderWaitShip).
218         Update("type", consts.OrderWaitReceive)
219     if res.Error != nil { return false, res.Error }
220     return res.RowsAffected > 0, nil
221 }
```

Listing 5: internal/order/repo.go:215–223 ShipOrder DAO

- WHERE type=WaitShip 兜住所有并发：第二次重发只会 RowsAffected=0，service 层返回 ErrInvalidOrderStateTransition。
- 这条 SQL 是状态机推进的**原子保证**：service 层就算被绕过，DB 仍能拒绝非法推进。
- 退款的 WHERE type IN (?) 同理—多 from 列表也能在 SQL 层一次拦截。

第 3 层：outbox 事件解耦下游服务 i

```
36 func (s *ShippingSrv) ShipOrder(ctx context.Context, orderNum uint64,  
37   trackingNo, carrier string) error {  
38   // ... 校验状态 + 取订单 ...  
39   return baseDao.DB.Transaction(func(tx *gorm.DB) error {  
40     ok, err := NewOrderDaoByDB(tx).ShipOrder(orderNum)  
41     if err != nil { return err }  
42     if !ok { return ErrInvalidOrderStateTransition }  
43     return outbox.NewOutboxDaoByDB(tx).Insert(  
44       "order", "OrderShipped", "order.shipped", order.ID,  
45       events.OrderShipped{ /* ... */ TrackingNo: trackingNo, Carrier: carrier})  
46   })  
47 }
```

Listing 6: internal/order/shipping.go:36–67 ShipOrder 主体（节选）

第 3 层：outbox 事件解耦下游服务 ii

- 状态变更 + outbox 写一个事务原子—不会出现”状态变了但事件没发”。
- 下游消费者：物流系统（落 tracking 表） / 推送服务（发”已发货”通知） / 平台 cron（启 7d 自动确认倒计时）。
- 任意下游挂掉**不影响主链路**：主链路只写 outbox，publisher 异步派送。

**业务边界：deck 09 不做什么（履约
gap 路线图）**

业务边界：deck 09 范围之外的 6 件事

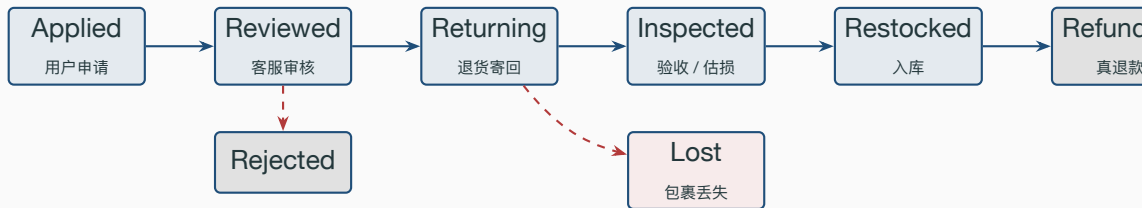
- 不直接物流商：当前 order.shipped 事件里带 tracking_no / carrier，但没有 **logistics_order** 子表—物流签收 / 拒收回写主表是路线图。
- 不接售后工单：RequestRefund / ApproveRefund / RejectRefund 只动 order 主表的状态机，没有 **aftersales** 工单表—退货寄回 / 验收 / 入库 / 估损扣减全部路线图。
- 不接评价审核：Completed 后无评价表，”自动好评”标记 (Auto: true) 透传给路线图的评价服务。
- 不发货通知给商家：商家工作台路线图，现版本商家凭后台 admin 接口看订单。
- 不接发票 / 海外清关 / 多币种：跨境单的清关 / 汇率快照 / 关税字段路线图。
- 不接钱包真打款：ApproveRefund 只写 order.refunded 事件，TxID 占位，下游 wallet 服务消费才真退钱；事件载荷另携带 promo_rule_id / promo_discount_cents，满减预算退还由 promo 侧消费同一事件异步完成 (promo_release 幂等台账兜重复投递)。

业务边界明确是专业表达的一部分—比”以为自己做了”更值钱。

路线图 1: logistics_order 子表一为什么不入主表

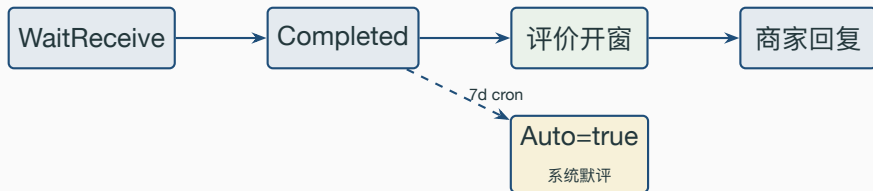
- 一个订单可能对应**多个物流包裹**：跨仓拆单 / 大件分箱 / 预售分批，必须 1:N。
- 物流子状态机（揽收 / 转运 / 派送 / 签收 / 拒收）节奏远高于主表 7 态更新频率。
- 物流商有 5+ 家（顺丰 / 京东 / 圆通 / EMS / 极兔），**每家 webhook JSON 不一样**，要 adapter 层抹平。
- 主表只保留 order_num，物流字段下沉到 logistics_order(carrier, tracking_no, status, ...)。
- **业务粒度切分**：高频物流节点（每条物流更新）和低频订单状态（7 态切换）分两张表写，避免互相阻塞。

路线图 2: aftersales 工单一售后完整生命周期



- 工单 6 正常态 + 2 异常态 (Rejected / Lost), 独立于主表 7 态运行。
- 工单只在 **Restocked** → **Refunded** 这一步反向回写主表 (推 Refunded)。
- **Inspected** 估损是工单和单纯退款最大的差别: 钱不一定全退 (货损 / 缺件按比例扣)。

路线图 3：评价表—挂在哪条状态边上



- 评价窗口只在订单到 Completed 后开启—避免”没收到货就先打差评”被薅。
- Auto: true (cron 兜底) → 平台默认好评，评价服务自行决定是否计入商家评分。
- Refunding 期间评价系统冻结：不允许写新评价，已写的按业务规则隐藏。

现状 vs 路线图：一张图看清边界

订单 7 态机

已实现

Outbox 事件

已实现

RMQ+Cron 关单

已实现（已加固）

7d 自动确认

已实现

Saga 释放预占

已实现

旧别名清退

已完成（已删）

logistics 子表

路线图

aftersales 工单

路线图

评价 / 默评

路线图

商家工作台

路线图

钱包真打款

路线图

多币种 / 跨境

路线图

- 上 6 项已落地（其中关单 cron 经历过一次表达式陷阱并已加固），下 6 项是路线图。
- gomall MVP 聚焦交易闭环—状态机推到 7 态，下游服务由 outbox 解耦后续逐步落地。

兼容性：3 → 7 零停机迁移

零停机迁移：DB 不动 + 别名兼容

- 硬约束：DB 有数百万存量 order，type 字段是 tinyint，改值需 ALTER + UPDATE 全表扫，**停机不可接受**。
- 老旧客户端（小程序 / Android / 历史 API）还在用旧字段名 UnPaid / Paid / Cancelled 拼前端字典。
- 迁移策略：
 - DB 数值**完全不动**：1 = WaitPay（旧名 UnPaid），语义对齐。
 - 迁移期旧名以 Deprecated 常量别名共存，老调用方编译不破。
 - 旧前端字典 OrderTypeMap 迁移期并存，转译同一组 int。
- 编译期校验：// Deprecated：让 staticcheck / golangci-lint warn 出来，新代码不写老名。
- 收尾：调用方全部切换后，别名块与 OrderTypeMap **已整体删除**，现行字典只剩 OrderStateMap。

常量层别名 + 编译期校验 + 按期清退 = **零停机 / 零回归**的迁移。

业务 Q&A (上)

Q1: 为什么 30min 关单不能由商家自定义?

A: 平台规则统一 = 用户预期统一 = 客服话术统一。商家自定义会导致客诉口径混乱 (路线图: 仅大件 / 预售类可商家配)。

Q2: 当初的 cron 表达式陷阱为什么没造成线上事故? 现在怎么收口的?

A: 靠 CloseOrderWithCheck 的 WHERE type=1 条件 UPDATE 兜底幂等 + 释放 reserved 是 Lua 幂等 + 关单不可逆, 所以误写期间最坏只是 60x 无效 DB 空跑、不丢单不超卖; 现已改用 @every 5m 从写法上消除秒位歧义。

Q3: 用户付了 3 天商家没发货, 怎么自动处理?

A: 当前靠客服 trigger RequestRefund (from=WaitShip 合法); 路线图加 cron 扫 WaitShip+48h 自动 Refunding。

Q4: 7d 自动确认收货对商家公平吗?

A: 业界共识 (淘宝实物 15d / 京东 7d / 拼多多 7d)。Completed 后仍可发起退款 → Refunding, 商家利益不会被一次性结算锁死。

Q5: 用户能撤回“已确认收货”吗?

A: 不能 (状态机不允许 Completed → WaitReceive 回退)。但走 Completed → Refunding 售后路径仍可退款。

业务 Q&A (下)

Q6: 商家发货 24h 后又想撤销发货，能吗？

A: 不能 (WaitReceive → WaitShip 不存在)。商家联系客服走用户全额退款流程。

Q7: 物流单号不入表，下游怎么知道？

A: order.shipped 事件 payload 带 tracking_no / carrier，下游物流服务自行落表；标准 outbox 解耦。

Q8: 退款多久能到账？

A: 状态机本身瞬时切换 (Refunding → Refunded)；真打款 1-15d 由下游钱包 / 支付通道决定 (路线图)。

Q9: 商家在用户申请退款期间能继续发货吗？

A: 不能。ShipOrder DAO 的 WHERE type=WaitShip 不会匹配 Refunding 行，RowsAffected=0 直接拒。

Q10: 3 态 → 7 态要发停机公告吗？

A: 不用。DB 数值 0 变更，迁移期常量层 alias 兜住编译，调用方切完后别名已删。

Q11: deck 11 讲的 outbox 和本 deck 矛盾吗？

A: 不矛盾。deck 11 讲 outbox 模式本身，本 deck 是应用方：每个状态切换都走 outbox 解耦下游。

Q12: 订单链路压测数字怎么对应业务承诺？

A: /orders/create 50K RPS / p95 2.33ms 对应 P0 SLO (99.95% / p99<500ms) 留 200x 余量；/orders/list 58K RPS 对应 P1 SLO。

代码位置一览

状态机表 + 工具函数

状态常量 (旧别名已删)

OrderStateMap 中文名

ShipOrder service

ConfirmReceive service

RequestRefund service

ApproveRefund service

RejectRefund service

CancelUnpaidOrder 共享入口

满减预算退还消费侧 (异步)

CloseOrderWithCheck DAO

ShipOrder DAO

ConfirmReceive DAO

RequestRefund DAO

GetTimeoutWaitReceive DAO

Cron 自动确认收货

Cron 注册 (现行 @every 5m)

RMQ 延迟队列 30min

业务码 70001 / 70002 / 60002